

XML850R_WM

Proposed source, changes highlighted | SGSRC/QRPGLESRC | generated 2026-08-26 | NOT applied to the IBM i

■ added or replaced ■ removed (struck through) white = unchanged

Change IDs in this member: C4, C7, C8, C10, C11, C13, C14, C15, C16, C17, C18, C19, C20

Line numbers count the proposed source only; removed lines are unnumbered.

```
1      H Debug(*Yes) TruncNbr(*Yes) AlwNull(*UsrCtl)
2      H DftActGrp(*No) ActGrp(*Caller)
3      H Option(*SrcStmt : *NoDebugIO) ExprOpts(*ResDecPos)
4      H BndDir('SGSRC/XML850R_WM')
5      //*****
6      // Author          - Chris Hutchinson / Pro I.T. Resource Group, Inc. *
7      //                  chutchinson@proitrg.com | 800-545-7630 *
8      //                  www.proitrg.com | 336-420-0329 *
9      // Program Name    - XML850R_WM *
10     // Program Date    - 09/17/2020 *
11     // Source Library   - QRPGLSRC/SGSRC *
12     // Object Library   - SGPGM *
13     // Program Descr    - Process incoming cXML order transactions for *
14     //                  Coupa - new orders only - change requests *
15     //                  have not been implemented yet on Coupa's side. *
16     //                  Coupa is currently only used by Waste Management *
17     //                  for orders and invoices. *
18     // *
19     // NOTE: Elements with attributes were broken down as on page 67 *
20     // of Scott Klement's "Web Services and XML for RPGers.pdf" *
21     // at 'https://www.scottklement.com/presentations/Web%20 *
22     // Services%20and%20XML%20for%20RPGers.pdf' *
23     // *
24     //*****
25     // *
26     // MODIFICATIONS *
27     // *
28     // *
29     //*****
30
31     // HarrisData Customer Master File
32     FHDCUST  IF  E          K Disk
33
34     // HarrisData Item Master File
35     FHDIMST  IF  E          K Disk
36
37     // HarrisData Item Plant Master File
38     FHDIPLT  IF  E          K Disk
39
40     // HarrisData Item Warehouse Master File
41     FHDIWHS  IF  E          K Disk
42
43     // HarrisData Order Flags Master
```

```

44      FHDORFG      IF      E              K Disk
45
46      // HarrisData Ship Via Master
47      FHDSHPV      IF      E              K Disk
48
49      // HarrisData Cost Summary Master File
50      FHDMCMM      IF      E              K Disk
51
52      // HarrisData Tax Definition File
53      FHDTAXD      IF      E              K Disk
54
55      // HarrisData Order Header Work File
56      FOEHDWK      UF A E              K Disk
57
58      // HarrisData Order Detail Work File
59      FOEDTWK      O      E              Disk      Block(*NO)
60
61      // HarrisData Order Comment Work File
62      FOECMWK      UF A E              K Disk      Block(*NO)
63
64      *Arpeggio SFTP Software - Server Definition File
65      * (for download path)
66      FAFSVR1P      IF      E              K Disk
67
68      // XML Orders Header Master
69      FXMLOEHD      O      E              Disk      Block(*NO)
70
71      // XML Orders Header Master
72      FXMLOEDT      O      E              Disk      Block(*NO)
73
74      * Program Fields *
75      D NumParams      s              1s 0
76      D ProcessMode      s              1a
77      D filename      s              256a
78      D ifsFile      s              256a
79      D Length      s              3s 0
80      D RestofLen      s              3s 0
81      D orderCounter      s              2s 0
82      D lineCounter      s              3s 0
83      D BuyerOrdLn      s              5s 0
84      D TempID      s              12a              OEHDWK/H1SHTO
85      D TempDate      s              d      DatFmt(*ISO)
86      - C10 D AddDays      s              7s 0
87      + C10 D AddDays      s              5s 0
88      D DaysOff      s              7s 0
89      D WorkDate      s              1a
90      D TodayISO      s              d      DatFmt(*ISO)
91      D LastDt1Date      s              d      DatFmt(*ISO)
92      D posstart      s              3s 0
93      D X      s              5s 0
94      D F      s              2s 0
95      D DFL      s              1a      Dim(20)
96      D APID      s              2a      Inz('OE')
97      D HexHnd      s              64a
98      D FGORTY      s              1a

```

98	D Day_of_Week	s	1s 0
99	D HDPCust#	s	7s 0
100	D xmlMode	s	1a
101	D xmlFASetID	s	10a
102	D xmlFATransID	s	10a
103	D OrdTotal	s	11p 2
104	D DQ#NAM	s	10a Inz('OEDTAQ')
105	D DQ#LIB	s	10a Inz('*LIBL')
106	D DQ#LEN	s	5s 0 Inz(81)
107	D GiftMSG	s	256a
108	D ShipToFnd	s	1a Inz('N')
109	D STTxCd	s	1a
110	D STPlaceId	s	20a
111	D STName1	s	30a
112	D STName2	s	30a
113	D STNameFL	s	30a
114	D STAdr1	s	30a
115	D STAdr2	s	30a
116	D STAdr3	s	30a
117	D STCity	s	30a
118	D STState	s	2a
119	D STZip	s	10a
120	D STCountry	s	3a
121	D STEmail	s	100a
122	D STPhone	s	15a
123	D BillToNum	s	7s 0
124	D BTPlaceId	s	20a
125	D BTName1	s	30a
126	D BTName2	s	30a
127	D BTAdr1	s	30a
128	D BTAdr2	s	30a
129	D BTAdr3	s	30a
130	D BTCity	s	30a
131	D BTState	s	2a
132	D BTZip	s	10a
133	D BTCntry	s	3a
134	D BTEmail	s	100a
135	D BTPhone	s	15a
136	D CSPlaceId	s	20a
137	D CSName1	s	30a
138	D CSName2	s	30a
139	D CSAdr1	s	30a
140	D CSAdr2	s	30a
141	D CSAdr3	s	30a
142	D CSCity	s	30a
143	D CSState	s	2a
144	D CSZip	s	10a
145	D CSEmail	s	100a
146	D CSPhone	s	15a
147	D IVPlaceId	s	20a
148	D IVName1	s	30a
149	D IVName2	s	30a
150	D IVAdr1	s	30a
151	D IVAdr2	s	30a
152	D IVAdr3	s	30a

153		D IVCity	s	30a	
154		D IVState	s	2a	
155		D IVZip	s	10a	
156		D IVEmail	s	100a	
157		D IVPhone	s	15a	
158		D UsePath	s	15a	
159		D Cmd	s	500a	
160		D CmdLen	s	15p 5	
161		D DropShip	s	7s 0	
162		D FAPath	s	155a	Varying
163		D FAFile	s	100a	Varying
164		D FAFull	s	256a	Varying
165		D RightNow	s	Z	
166		D Avail	s	13p 4	
167		D from	s	100a	
168		D To	s	100a	
169		D replyTo	s	100a	
170		D subject	s	100a	
171		D text	s	9999a	
172		D distribution	s	100a	
173		D email	s	0	class(*java:RPGMail)
174		D Alert	s	10a	
175		D AlertSubj	s	200a	
176		D AlertMsg	s	9999a	
177		D pass	s	10i 0	
178		D orderAction	s	8a	
179		D StockItem	s	1a	
180		D NullInd	s	2b 0	
181		D UDCCDays	s	2s 0	
182	+	C10	D WkClass	s	Like(IMPCLS)
183	+	C10	D CustClass	s	Like(CMCCLS)
184	+	C10	D LeadDays	s	5s 1
185	+	C10	D ReqDays	s	5s 0
186	+	C10	D ProcessedItems	s	3s 0
187					
188		D XMLorderId	s	20a	
189		D LastOrder	s	8s 0	
190		D CurDate	s	D	DatFmt(*ISO)
191		D CurTime	s	T	
192		D Transact	s	10a	
193		D HWLinCnt	s	5s 0	
194		D lineTotal	s	10i 0	
195		D OrderTotal	s	10i 0	
196		D CustomItem	s	1a	
197		D DebugFlag	s	1a	
198		D SaveDesc	s	300a	
199		D CharPos	s	3s 0	
200					
201		D 01IMDS	s	30a	
202		D 01BITEM	s	35a	
203		D 01VITEM	s	35a	
204		D 01ITEM	s	15a	
205		D 01CUOM	s	3a	Inz('EA')
206		D 01PUOM	s	3a	Inz('EA')
207		D 01QUOM	s	3a	Inz('EA')

208	D 01ORL#	s	3P 0	
209	D 01QORD	s	13p 4	
210	D 01RQDTA	s	25a	
211	D 01RQDTD	s	D	DatFmt(*ISO-)
212	D 01COST	s	13p 5	
213	D 01SLPR	s	13p 5	
214	D 01SLPRXLT	s	13a	
215	D 01SVSV	s	2a	
216	D 01CNCDT	s	8a	
217	D 01DLVDT	s	8a	
218				
219	D ErrFlag	s	1a	
220	D ErrDescr	s	100a	
221	D RecsWrtn	s	1a	
222				
223				
224	D HWPO#	s	22a	OEHDWK/H1ORRF
225	D HWOrdDate	s	25a	yyyymmdd
226	D HWMerchDpt	s	10a	
227	D HWShpToID	s	15a	OEHDWK/H1SHTO
228	D HWBillToID	s	12a	OEHDWK/H1BLTO
229	D HWLineCnt	s	3s 0	
230	D HWCustID	s	12a	OEHDWK/H1BLTO
231	D HWInvToID	s	12a	OEHDWK/H1BLTO
232	D HWShipCd	s	15a	OEHDWK/H1SVSV-H1SVDS
233	D HWControl	s	5a	
234	D HWSlsDiv	s	10a	
235	D HWCustOrd	s	15a	???????
236	D HWBuyCntrct	s	10a	???????
237	D HWReqShpD	s	10a	
238	D HWPOType	s	2a	
239	D HWGftMSG	s	256a	
240				
241	D xmLOCTL	s	8p 0	
242	D xmLFRT	s	9p 2	
243	D xmLCHRG	s	9p 2	
244	D xmLUPDTOT	s	1a	
245				
246	D CMSvSv	s	2a	
247				
248	D CustOrdLn	s	3s 0	
249	D CustRqDt	s	7s 0	
250	D CustDesc	s	30a	
251	D CustCmt	s	60a	
252	D CustQOrd	s	13s 4	
253	D CustSlPr	s	13s 5	
254				
255	// XML Acknowledgement Variables *			
256				
257	// Customer#			
258	D XMLACKCUST	s	20A	
259	// Acknowledgement Date (YYYY-MM-DD)			
260	D XMLACKDATE	s	D	datfmt(*ISO)
261	// Ack. Order ID			
262	D XMLACKORDR	s	20A	

```

263 // Ack. Customer PO#
264 D XMLCUSTPO      s      22A
265 // Ack. Type
266 D XMLACKTYPE     s      6A
267 // Customer PO Date
268 D XMLCUSTPD      s      D   datfmt(*ISO)
269
270
271 D HOEUMC_W       PR      ExtPgm('HOEUMC_W')
272 D HexHnd         64a
273 D xmlDBID        2a
274 D xmlOCTL        8p 0
275 D xmlFRT         9p 2
276 D xmlCHRG        9p 2
277 D xmlUPDTOT      1a
278
279 D Ack            PR      ExtPgm('XMLACKR')
280 D xmlRetailer    15a
281 D HWPO#         22a
282 D ErrFlag        1a
283 D ErrDescr       100a  Options(*NoPass)
284
285 D XML850R_WM     PR      ExtPgm('XML850R_WM')
286 D xmlDBID        2a
287 D xmlRetailer    15a
288 D xmlFile        100a
289 D xmlSvrDfn      10a  Options(*NOPASS)
290
291 D XML850R_WM     PI
292 D xmlDBID        2a
293 D xmlRetailer    15a
294 D xmlFile        100a
295 D xmlSvrDfn      10a  Options(*NOPASS)
296
297 D error          DS      LikeDS(Error_Info)
298
299 D DTAQ           E DS      ExtName(OEDTAQPL)
300
301
302 // Prototype for Send Data Queue (QSNDDTAQ) API
303 D QSNDDTAQ       PR      ExtPgm('QSNDDTAQ')
304 D DQ#NAM         10a  Const
305 D DQ#LIB         10a  Const
306 D DQ#LEN         5p 0  Const
307 D Data           32766a  Options(*varsize)
308
309 // QCmdExc Parameters *
310 D ExecCmd        PR      EXTPGM('QCMDEXC')
311 D Cmd            500a
312 D CmdLen         15p 5
313
314 // Prototypes for RPGMail *
315 /copy RPGMail/qsource,RPGMailCP
316 /copy RPGMail/qsource>ErrorCP
317

```

```

318 // QMHSNDPM prototype + work fields - log alert send failures (3c)
319 dcl-pr SndPgMMsg ExtPgM('QMHSNDPM');
320     p_MsgID      char(7)      const;
321     p_MsgFile    char(20)     const;
322     p_MsgData    char(32767)  const options(*varsize);
323     p_MsgDataLen int(10)      const;
324     p_MsgType    char(10)     const;
325     p_CallStkEnt char(10)     const;
326     p_CallStkCnt int(10)      const;
327     p_MsgKey     char(4);
328     p_ErrCode    char(32767)  options(*varsize);
329 end-pr;
330
331 dcl-s MsgKey char(4);
332 dcl-s LogTxt char(512);
333 dcl-ds ApiErr qualified;
334     BytesProv int(10) inz(8);
335     BytesAvail int(10) inz(0);
336 end-ds;
337
338
339
340
341
342 //*****
343 /** XML Data TEMPLATES **
344 //*****
345
346 //-----
347 //| Header Template |
348 //-----
349
350 dcl-ds Sender_T Qualified Template ;
351     dcl-ds Credential likeDS(Credential_T) ;
352     UserAgent Char(20) ;
353 // end-ds ;      ---> NOT specified after a DS is declared with likeDS!!!
354 end-ds Sender_T ;
355
356 dcl-ds Credential_T Qualified Template ;
357     domain      Char(20) ;
358     Identity     Char(20) ;
359     SharedSecret Char(15) ;
360 end-ds;
361
362 //*****
363 // XML Data Structure Definitions *
364 //*****
365 dcl-ds cXML Qualified ;
366     payloadID    Char(50) ;
367     version      Char(10) ;
368
369 dcl-ds Header ;
370
371     dcl-ds From ;
372     dcl-ds Credential ;

```

```

373         domain Char(20) ;
374         Identity Char(20) ;
375     end-ds Credential ;
376 end-ds From ;
377
378 dcl-ds To ;
379     dcl-ds Credential ;
380         domain Char(20) ;
381         Identity Char(20) ;
382     end-ds Credential ;
383 end-ds To ;
384
385 dcl-ds Sender ;
386     dcl-ds Credential likeDS(Credential_T) inz(*likeDS) ;
387 //     end-ds Credential ;      ---> Not used on a data structure declared with likeDS!!!
388 end-ds Sender ;
389
390 end-ds Header ;
391
392 dcl-ds Request ;
393     deploymentMode Char(10) ;
394
395     dcl-ds OrderRequest Dim(50);
396     dcl-ds OrderRequestHeader ;
397         orderID Char(15) ;
398         numoforderID Int(10) ;
399         orderDate Char(25) ;
400         type Char(10) ;
401         orderType Char(8) ;
402         // orderVersion Zoned(2:0) ;
403     dcl-ds Total ;
404         dcl-ds Money ;
405 //         alternateAmount Char(10) ;      // Was Zoned(8:2) but they don't send it
406 //         alternateCurrency Char(10) ;
407         currency Char(3) ;
408         dsvalue Char(50) ;
409     end-ds Money ;
410 end-ds Total ;
411 dcl-ds ShipTo ;
412     dcl-ds Address ;
413         isoCountryCode Char(5) ;
414         addressID Char(25) ;
415     dcl-ds Name ;
416         xml_lang Char(2);      // Was xml:lang; Works w/case convert
417         dsvalue Char(50);
418     end-ds Name ;
419     dcl-ds PostalAddress ;
420         name Char(10) ;
421         // DeliverTo Char(30) Dim(2) ;
422         DeliverTo Char(30) ;
423         Street Char(30) ;
424         City Char(15) ;
425         State Char(2) ;
426         PostalCode Char(10) ;
427     dcl-ds Country ;

```



```

428         isoCountryCode Char(2) ;
429         dsvalue Char(50) ;
430     end-ds Country ;
431 end-ds PostalAddress ;
432 dcl-ds Email ;
433     name Char(10) ;
434     // preferredLang Char(5) ;
435     dsvalue Char(50) ;
436 end-ds Email ;
437 dcl-ds Phone ;
438     name Char(10);
439     dcl-ds TelephoneNumber ;
440         dcl-ds CountryCode ;
441             isoCountryCode Char(2) ;
442             dsvalue Char(1) ;
443         end-ds CountryCode ;
444         AreaOrCityCode Zoned(3:0) ;
445         Number Zoned(7:0) ;
446     end-ds TelephoneNumber ;
447 end-ds Phone ;
448 dcl-ds Fax ;
449     name Char(10) ;
450     dcl-ds TelephoneNumber ;
451         dcl-ds CountryCode ;
452             isoCountryCode Char(2) ;
453             dsvalue Char(1) ;
454         end-ds CountryCode ;
455         AreaOrCityCode Zoned(3:0) ;
456         Number Zoned(7:0) ;
457     end-ds TelephoneNumber ;
458 end-ds Fax ;
459 end-ds Address;
460 end-ds ShipTo;
461
462 dcl-ds BillTo ;
463     dcl-ds Address ;
464         isoCountryCode Char(2) ;
465         addressID Char(4) ;
466         dcl-ds Name ;
467             xml_lang Char(2) ;
468             dsvalue Char(32) ;
469         end-ds Name ;
470         dcl-ds PostalAddress ;
471             name Char(30) ;
472             Street Char(50) ;
473             City Char(30) ;
474             State Char(2) ;
475             PostalCode Char(10) ;
476         dcl-ds Country ;
477             isoCountryCode Char(2) ;
478             dsvalue Char(25) ;
479         end-ds Country ;
480     end-ds PostalAddress ;
481 end-ds Address ;
482 end-ds BillTo ;

```

```

483      dcl-ds Shipping ;
484      dcl-ds Money ;
485          currency      Char(3) ;
486          dsvalue       Zoned(9:2) ;
487      end-ds Money ;
488  end-ds Shipping ;
489  dcl-ds PaymentTerm ;
490      payInNumberOfDays Zoned(3:0) ;
491  end-ds PaymentTerm ;
492  Comments      Char(250) ;
493  dcl-ds Contact;
494      role      Char(25) ;
495      Name      Char(15) ;
496      dcl-ds Email ;
497          name   Char(50) ;
498      End-ds Email ;
499  end-ds Contact ;
500  end-ds OrderRequestHeader ;
501
502  dcl-ds ItemOut Dim(400) ;
503      quantity      Char(13) ;
504      lineNumber     Zoned(5:0) ;
505      dcl-ds ItemID ;
506          SupplierPartID      Char(15) ;
507          numofSupplierPartID  Int(10) ;
508          SupplierPartAuxiliaryID Char(15) ;
509      end-ds ItemID ;
510
511  dcl-ds ItemDetail ;
512      dcl-ds UnitPrice ;
513          dcl-ds Money ;
514              currency      Char(3) ;
515              dsvalue       Char(13) ;
516          end-ds Money ;
517      end-ds UnitPrice ;
518      dcl-ds Description ;
519          xml_lang   Char(2) ;
520          dsvalue    Char(300) ;
521      end-ds Description;
522      UnitOfMeasure Char(2) ;
523      dcl-ds Classification ;
524          domain      Char(10) ;
525          Classification Char(10) ;
526      end-ds Classification ;
527      dcl-ds Extrinsic ;
528          name        char(25) ;
529          // dsValue  zoned(9:0) ;
530          dsValue     char(25) ;
531      end-ds Extrinsic ;
532
533  end-ds ItemDetail ;
534
535  dcl-ds Distribution ;
536      dcl-ds Accounting ;
537          name      Char(150) ;

```

```

538         dcl-ds Segment Dim(4) ;
539         type          char(25) ;
540         id            char(10) ;
541         description   char(5) ;
542     end-ds Segment ;
543 end-ds Accounting ;
544 dcl-ds Charge ;
545     dcl-ds Money ;
546     currency      char(3) ;
547     dsvalue       zoned(13:4) ;
548     end-ds Money ;
549     end-ds Charge ;
550 end-ds Distribution ;
551 end-ds ItemOut ;
552
553     end-ds OrderRequest ;
554 end-ds Request ;
555 end-ds cXML ;
556
557
558
559 // Constants *
560 D Up          C          'ABCDEFGHJKLMNOPQRSTUVWXYZ'
561 D Lo          C          'abcdefghijklmnopqrstuvwxyz'
562 D CrLf       C          'Const(x'0d25')
563
564 // Search/Replace Constants
565 - C19 D apos C '&apos;'
566 + C19 D apos C '&apos;';
567 - C19 D quot C '&quot;'
568 + C19 D quot C '&quot;';
569 - C19 D amper C '&amp;'
570 + C19 D amper C '&amp;';
571 - C19 D lt C '&lt;'
572 + C19 D lt C '&lt;';
573 - C19 D gt C '&gt;'
574 + C19 D gt C '&gt;';
575
576 // ----- Main Module -----*
577
578 /Free
579
580 // Prepare email for alerts
581 Monitor;
582     RPGMail_init();
583 On-error;
584     error = Error_catch();
585 EndMon;
586
587 // Reset the error flag.
588 ErrFlag = 'N';
589
590 // Check the number of parameters. 4 parameters mean this is a
591 // production run. The 4th parameter is the server definition

```

```

588 // used by ARPEGGIOL. This is not passed if this is a test run
589 // of an archived inbound order.
590 NumParams = %Parms;
591 If NumParams < 4;
592     // Testing a /orders/archived inbound/wastemgmt/{filename} order
593     ProcessMode = 'T';
594 Else;
595     // Executing a production run of a wastemgmt order
596     ProcessMode = 'L';
597 EndIf;
598
599 Exsr getXMLData;
600
601 // Process UP TO 50 orders per cXML file (per the "Dim"
602 // statement on the SGOrder Data Structure spec)
603 Exsr loadDB2Data;
604
605 // If this is a production order, archive the order.
606 If ProcessMode = 'L';
607     Exsr exitProgram;
608 EndIf;
609
610 *InLR = *On;
611
612
613 //-----*
614 // getXMLData Subroutine will pull all XML variables out and *
615 // put them into their appropriate production data fields *
616 //-----*
617
618
619 BegSr getXMLData;
620
621     BillToNum = 5000;
622     H1B1To    = 5000;
623
624     usepath = XMLRetailer;
625
626     // Pull extraneous data required based on the trading
627     // partner being processed...
628     Exec SQL
629         Select Substr(UDCDESC3,1,1), Dec(UDCDESC2,7,0)
630             Into :xmlMode, :HDPCust#
631             From ProITRG/UDCDETAIL
632             Where UDCSYSTEMD = 'XML' And UDCKEY = :XMLRetailer;
633
634     If SQLCod <> 0;
635         // Notify somebody of an error trying to process :XMLRetailer!!!
636         // Don't know how this could happen if the user got this far.
637         Alert = 'XMLNOCUST';
638         Select;
639         When ProcessMode = 'T';
640             AlertSubj = '*** Testing Archived Inbound Order file ' +
641                 %Trim(xmlFile) + ' ***';
642         When xmlMode = 'T';

```

```

643     AlertSubj = '*** Test Order file ' + %Trim(xmlFile) +
644         ' sent in by ' + %Trim(xmlRetailer) +
645         '***';
646 Other;
647     AlertSubj = 'Waste Management Order ' + %Trim(xmlFile) +
648         ' Alert';
649 EndSL;
650
651 AlertMsg = %Trim(AlertMsg) +
652     '--> This file cannot be processed. ' +
653     'No HD primary customer number in the User' +
654     ' Defined Codes file for ' +
655     %Trim(xmlretailer) + '.';
656 Exsr Notify ;
657 *InLR = *On;
658 Return;
659 EndIf;
660
661 // Pull the path used for the download to extract the filename
662 // if this is a production order.
663 If ProcessMode = 'L';
664     Chain xmlSvrDfn AFSVR1P ;
665
666     Length = %len(%Trim(SFDWNFLR)) ;
667     If Length = 0 ;
668         Alert = 'XMLORDERS';
669         Alert = 'XMLORDERR';
670         AlertSubj = 'Waste Management Order file ' + %Trim(xmlFile) +
671             ' Alert';
672
673         AlertMsg = 'This file cannot be processed. ' +
674             'No path in the ' + %Trim(xmlSvrDfn) +
675             ' server definition in ARPSFTP.' ;
676         Exsr Notify ;
677         *InLR = *On ;
678         Return ;
679     EndIf;
680 EndIf;
681
682 // ProcessMode takes precedence over xmlMode because
683 // ProcessMode is what we're doing at a higher level
684 // ...production order a testing an order from archived
685 // inbound. xmlMode is which we're processing for the
686 // retailer - a test order they sent or a production order
687 // they sent.
688 Select;
689 When ProcessMode = 'T' Or xmlMode = 'T';
690     filename = xmlFile;
691
692 Other;
693     RestofLen = %len(xmlFile) - Length;
694     ifsFile = %Subst(xmlFile:Length + 1:RestofLen);
695     filename = '/orders/inbound/' +
696         %Trim(xmlRetailer) + '/' + %Trim(ifsFile);

```

```

697      EndSL;
698
699      XML-INTO cXML %xml(%Trim(filename):
700          'case=convert doc=file datasubf=dsvalue +
701          allowextra=yes allowmissing=yes +
702          countprefix=numof ns=merge');
703
704
705      EndSr;
706
707      //-----*
708      // loadDB2Data subroutine will populate all the data required *
709      // for HarrisDatas' normal order processing. *
710      //-----*
711
712      BegSr loadDB2Data;
713
714      orderTotal = cXML.Request.OrderRequest(1).
715          OrderRequestHeader.numofOrderID;
716
717      For orderCounter = 1 to orderTotal;
718
719          // Check to see if this is a Change or Cancel request.
720          // If it is, just email customer service and don't
721          // process the order.
722          If %Trim(cXML.Request.OrderRequest(orderCounter).
723              OrderRequestHeader.type) = 'new' AND
724              (%Trim(cXML.Request.OrderRequest(orderCounter).
725                  OrderRequestHeader.orderType) = 'regular' OR
726                  %Trim(cXML.Request.OrderRequest(orderCounter).
727                      OrderRequestHeader.orderType) = *Blanks);
728
729              Exsr writeOrder;
730
731          Else;
732
733              // Notify Customer Service of Order Type we
734              // can't handle...
735
736              orderAction = cXML.Request.OrderRequest(orderCounter).
737                  OrderRequestHeader.orderType;
738
739              // Convert orderAction to uppercase with SQL
740              Exec SQL
741                  Set :orderAction = Upper(:orderAction);
742
743              Alert = 'ORDUPDATE';
744              AlertSubj = 'Waste Management Order Alert for file ' +
745                  %Trim(xmlFile) + '. Order type ' +
746                  %Trim(orderAction) +
747                  ' - PO# ' + %Trim(%Char(cXML.Request.
748                      OrderRequest(orderCounter).
749                      OrderRequestHeader.orderID)) ;
750              AlertMsg = 'Program XML850R_WM does not know how it ' +
751                  'should process this order type.';

```

```

752         Exsr Notify;
753
754     EndIf;
755
756     EndFor;
757
758 EndSr;
759 //-----*
760
761
762
763 //-----*
764 /* writeOrder loads the fields for the transaction header
765 //-----*
766 BegSr writeOrder;
767
768     Clear OEHDWKR;
769     Clear XMLOEHDR;
770
771     // Write the header for the order...
772     XMLOrderID = cXML.Request.OrderRequest(orderCounter).
773                 OrderRequestHeader.orderID;
774
775     Exsr NxtHDOrd;
776
777     HWPO# = cXML.Request.
778             OrderRequest(orderCounter).
779             OrderRequestHeader.orderID;
780     HWPO# = %XLate(Lo:Up:HWPO#);
781
782 - C18 ----- If HWPO# = '4100022893';
783 - C18 ----- DebugFlag = 'Y';
784 - C18 ----- EndIf;
785 + C18 // C18 - leftover debug, DebugFlag is never read
786 + C18 // If HWPO# = '4100022893';
787 + C18 // DebugFlag = 'Y';
788 + C18 // EndIf;
789
790     HWOrdDate = %Char(%Date(%Subst(cXML.Request.
791                                   OrderRequest(orderCounter).
792                                   OrderRequestHeader.orderDate:1:10):*ISO-));
793
794     If %Upper(%Subst(cXML.Request.OrderRequest(orderCounter).
795                     OrderRequestHeader.ShipTo.Address.addressID:1:7)) =
796         'ADBUYER';
797         HWShpToID = %Subst(cXML.Request.OrderRequest(orderCounter).
798                           OrderRequestHeader.ShipTo.Address.addressID:8:15);
799     Else;
800         HWShpToID = cXML.Request.
801                     OrderRequest(orderCounter).
802                     OrderRequestHeader.ShipTo.
803                     Address.addressID;
804     EndIf;
805
806 // STName1 = cXML.Request.

```

```

804      //      OrderRequest(orderCounter).
805      //      OrderRequestHeader.ShipTo.
806      //      Address.PostalAddress.Name;
807      STName1 = %Trim(cXML.Request.
808      OrderRequest(orderCounter).
809      OrderRequestHeader.ShipTo.
810      Address.Name.dsValue);
811      STName1 = %Trim(STName1) + ' - ' +
812      cXML.Request.
813      OrderRequest(orderCounter).
814      OrderRequestHeader.ShipTo.Address.addressID;
815      STName1 = %XLate(Lo:Up:STName1);
816      STAdr1 = cXML.Request.
817      OrderRequest(orderCounter).
818      OrderRequestHeader.ShipTo.
819      Address.PostalAddress.DeliverTo;
820      STAdr1 = %XLate(Lo:Up:STAdr1);
821      STAdr2 = cXML.Request.
822      OrderRequest(orderCounter).
823      OrderRequestHeader.ShipTo.
824      Address.PostalAddress.Street ;
825      STAdr2 = %XLate(Lo:Up:STAdr2);
826      STCity = cXML.Request.
827      OrderRequest(orderCounter).
828      OrderRequestHeader.ShipTo.
829      Address.PostalAddress.City;
830      STCity = %XLate(Lo:Up:STCity);
831      STState = cXML.Request.
832      OrderRequest(orderCounter).
833      OrderRequestHeader.ShipTo.
834      Address.PostalAddress.State;
835      STState = %XLate(Lo:Up:STState);
836      STZip = cXML.Request.
837      OrderRequest(orderCounter).
838      OrderRequestHeader.ShipTo.
839      Address.PostalAddress.PostalCode;
840      STCountry = cXML.Request.
841      OrderRequest(orderCounter).
842      OrderRequestHeader.ShipTo.
843      Address.PostalAddress.
844      Country.isoCountryCode;
845      STCountry = %XLate(Lo:Up:STCountry);
846      STEmail = cXML.Request.
847      OrderRequest(orderCounter).
848      OrderRequestHeader.ShipTo.
849      Address.Email.dsvalue;
850      STEmail = %XLate(Lo:Up:STEmail);
851
852
853      Monitor;
854      // All ShipTos won't provide a phone number
855
856      STPhone = %Char(cXML.Request.
857      OrderRequest(orderCounter).
858      OrderRequestHeader.ShipTo.

```



```

859             Address.Phone.TelephoneNumber.
860             AreaOrCityCode) +
861             %Char(cXML.Request.
862             OrderRequest(orderCounter).
863             OrderRequestHeader.ShipTo.
864             Address.Phone.TelephoneNumber.
865             Number);
866 On-Error;
867     STPhone = *Blanks;
868 EndMon;
869
870 BTName1 = cXML.Request.
871         OrderRequest(orderCounter).
872         OrderRequestHeader.BillTo.
873         Address.Name.dsvalue;
874 BTName1 = %XLate(Lo:Up:BTName1);
875 BTAdr1 = cXML.Request.
876         OrderRequest(orderCounter).
877         OrderRequestHeader.BillTo.
878         Address.PostalAddress.Street;
879 BTAdr1 = %XLate(Lo:Up:BTAdr1);
880 BTCity = cXML.Request.
881         OrderRequest(orderCounter).
882         OrderRequestHeader.BillTo.
883         Address.PostalAddress.City;
884 BTCity = %XLate(Lo:Up:BTCity);
885 BTState = cXML.Request.
886         OrderRequest(orderCounter).
887         OrderRequestHeader.BillTo.
888         Address.PostalAddress.State;
889 BTState = %XLate(Lo:Up:BTState);
890 BTZip = cXML.Request.
891         OrderRequest(orderCounter).
892         OrderRequestHeader.BillTo.
893         Address.PostalAddress.PostalCode;
894 BTCntry = cXML.Request.
895         OrderRequest(orderCounter).
896         OrderRequestHeader.BillTo.
897         Address.PostalAddress.
898         Country.isoCountryCode;
899 BTCntry = %XLate(Lo:Up:STCountry);
900
901 // Determine the ShipTo as follows...
902 // (Preferred)
903 // - Pull the Customer whose addressID in the cXML document
904 //   that matches the CMUDF4 field on HDCUST and has a BillTo
905 //   of :BillToNum
906 // (Secondary option)
907 // - If that doesn't exist, pull the Customer whose addressID
908 //   in the cXML document matches the CMUDF4 field on HDCUST
909 //   and use that BillTo and ShipTo
910 // (Tertiary option)
911 // - If neither of those exist, set up the order as a drop
912 //   ship with the information provided in the cXML document
913 //   and notify orders@screen-graphics.com of the issue.

```

```

914 ShipToFnd = 'N';
915
916
917 Exec SQL
918 Select CMCUST, CMSV, CMCTXC Into :H1ShTo, :CMSVSV, :STTxCd
919 From HDCUST
920 Where CMUDF4 = :HWSHpToID
921 And CMBLTO = :BillToNum;
922
923 If SQLCod = 0;
924 ShipToFnd = 'Y';
925 EndIf;
926
927 If ShipToFnd = 'N';
928 Exec SQL
929 Select CMCUST, CMBLTO, CMSV, CMCTXC
930 Into :H1ShTo, :H1B1To, :CMSVSV, :STTxCd
931 From HDCUST
932 Where CMUDF4 = :HWSHpToID;
933
934 If SQLCod = 0;
935 ShipToFnd = 'Y';
936 EndIf;
937 EndIf;
938
939 If ShipToFnd = 'N';
940 H1ShTo = BillToNum;
941
942 // Figure the Drop Ship Number for this Order
943 Exec SQL
944 Select DSNMBR Into :DropShip
945 From HDDSHP
946 Where DSVCF = 'C' And DSVNCS = :HDPCust#
947 And DSNAME = :STName1 And DSADR1 = :STADR1
948 And DSADR2 = :STADR2 And DSADR3 = :STADR3
949 And DSCITY = :STCITY And DSST = :STSTATE
950 And DSZIP = :STZIP And DSCTRY = :STCOUNTRY
951 And DSCNTC = :CMCNTC And DSCTTC = :CMCTTC
952 And DSLOC1 = :CMLOC1 And DSLOC2 = :CMLOC2
953 And DSLOC3 = :CMLOC3;
954
955 If SQLCod <> 0;
956
957 Exec SQL
958 Select Max(DSNMBR) Into :DropShip from HDDSHP;
959
960 DropShip += 1;
961
962 Exec SQL
963 Insert Into HDDSHP
964 (DSVCF, DSVNCS, DSNMBR, DSNAME, DSADR1, DSADR2, DSADR3,
965 DSCITY, DSST, DSZIP, DSCTRY, DSCNTC, DSCTTC, DSLOC1,
966 DSLOC2, DSLOC3, DSDATE, DSPURG, DSNAMEU, DSADR1U,
967 DSADR2U, DSADR3U, DSCITYU)
968 Values('C', :HDPCust#, :DropShip, :STName1, :STAdr1,

```

```

969      :STAdr2, :STAdr3, :STCity, :STState, :STZip,
970      :STCOUNTRY, :CMCNTC, :CMCTTC, :CMLOC1, :CMLOC2,
971      :CMLOC3, :H1RQDT, 'Y', UCASE(:STName1),
972      UCASE(:STADR1), UCASE(:STAdr2), UCASE(:STAdr3),
973      UCASE(:STCity));
974
975      H1DSHP = DropShip;
976
977      // *** Notify bbusch@screen-graphics.com of ***
978      // *** the new customer that needs to get ***
979      // *** set up using the drop ship info ***
980      Alert = 'XMLDrpShp';
981      Select;
982      When ProcessMode = 'T';
983          AlertSubj = 'Testing archive file ' +
984                    %Trim(xmlFile) + ' for Waste Management';
985      When xmlMode = 'T';
986          AlertSubj = 'Test file ' + %Trim(xmlFile) +
987                    'sent by Waste Management added a Drop Ship';
988      Other;
989          AlertSubj = 'Waste Management order added' +
990                    ' a Drop Ship';
991      EndSL;
992
993      AlertMsg = 'Waste Management file ' + %Trim(xmlFile)
994                + CrLf;
995      AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '
996                + %Char(LastOrder)
997                + CrLf;
998      AlertMsg = %Trim(AlertMsg) + 'Imported Order: '
999                + HWPO# + CrLf;
1000      AlertMsg = %Trim(AlertMsg) + 'New Drop Ship: '
1001                + %Char(DropShip) + CrLf + CrLf;
1002      AlertMsg = %Trim(AlertMsg) + 'Code: '
1003                + HWShpToID + CrLf;
1004      AlertMsg = %Trim(AlertMsg) + 'Address: '
1005                + %Trim(STName1) + CrLf;
1006      AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr1) +
1007                CrLf;
1008      AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr2) +
1009                CrLf;
1010      AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STAdr3) +
1011                CrLf;
1012      AlertMsg = %Trim(AlertMsg) + '          ' + %Trim(STCity) +
1013                ', ' + %Trim(STState) + ' ' + %Trim(STZip) +
1014                CrLf + CrLf;
1015      AlertMsg = %Trim(AlertMsg) + 'Issue: CMUDF4 info right?'
1016                + CrLf;
1017      Exsr Notify;
1018      Else;
1019
1020          H1DSHP = DropShip;
1021
1022      EndIf;
1023

```

```

1024         EndIf;
1025
1026         // Shipping charges for the freight field on the header
1027         H1Frt = cXML.Request.OrderRequest(orderCounter).OrderRequestHeader.
1028             Shipping.Money.dsvalue;
1029
1030
1031         CustomItem = 'N';
1032
1033         // Write the detail for the order...
1034 +      C13         // C13 - count the detail lines actually written
1035 +      C13         ProcessedItems = 0;
1036 +      C13
1037         For lineCounter = 1 to 500;
1038
1039             Clear OEDTWKR;
1040             Clear XMLOEDTR;
1041
1042             // If the line is not there it will throw an error, exit
1043             // at that point because all items have been processed.
1044
1045             Monitor;
1046
1047             BuyerOrdLn = cXML.Request.
1048                 OrderRequest(orderCounter).
1049                 ItemOut(lineCounter).
1050                 lineNumber;
1051
1052             // If BuyerOrdLn > 999;
1053             O1OrL# = lineCounter;
1054             // Else;
1055             // O1OrL# = BuyerOrdLn;
1056             // EndIf;
1057
1058             On-Error;
1059             Leave;
1060             EndMon;
1061
1062             O1VItem = cXML.Request.
1063                 OrderRequest(orderCounter).
1064                 ItemOut(lineCounter).ItemID.
1065                 SupplierPartID;
1066             O1VItem = %XLate(Lo:Up;O1VItem);
1067             O1ITEM = O1VITEM;
1068
1069             // Exit the loop if the last line was just processed...
1070             If O1VITEM = *Blanks;
1071                 Leave;
1072             EndIf;
1073
1074             O1ITEM = O1VITEM;
1075
1076             O1BItem = cXML.Request.
1077                 OrderRequest(orderCounter).
1078                 ItemOut(lineCounter).ItemID.

```

```

1079         SupplierPartID;
1080     01BItem = %XLate(Lo:Up:01BItem);
1081
1082     Monitor;
1083     01QOrd = %Dec(%XLATE('$*',':' ' :cXML.Request.
1084         OrderRequest(orderCounter).
1085         ItemOut(lineCounter).
1086         quantity) : 13 : 4);
1087 // On-Error 105;
1088 On-Error;
1089     01QOrd = 0;
1090 EndMon;
1091
1092 // Iter if the quantity is 0 AND the item is blank
1093 // because they are sending in orders without a
1094 // SupplierPartID that they want processed and we
1095 // don't want to assume that's the end of the order
1096 // and possibly miss something.
1097
1098 // If 01VItem = *Blanks And
1099 //     01QOrd = 0;
1100 // Iter;
1101 // EndIf;
1102
1103 01QUOM = 'EA';
1104
1105 // ***** //
1106 // If the item is NOT found on HDIMST it is to be processed as a //
1107 // custom, non-stock item ...per Bill Busch 10/15/2019 //
1108 // ***** //
1109
1110 // Description is longer than HD's OEORDT ODIMDS field, so
1111 // check to see if the description includes IMIMDS and, if
1112 // so write the remainder of the SAVEDESC after the ";" to
1113 // OECMWK.
1114 Chain 01ITEM HDIMST;
1115
1116 // Item is not on HDIMST so set it as a 'CUSTOM ITEM'
1117 If NOT %Found(HDIMST);
1118     CustomItem = 'Y';
1119 + C7 // C7 - a failed chain leaves the prior line's
1120 + C7 // IMPCLS in the record. Never read IMPCLS
1121 + C7 // directly again; use WkClass.
1122 + C7 WkClass = *Blanks;
1123     01OREC = 'N';
1124     01ITEM = 'CUSTOM ITEM';
1125     01PCLS = '9MIS';
1126     01STUM = 'EA';
1127     01PRUM = 'EA';
1128     01ORUF = 1.0000;
1129     01TAXC = 'Y';
1130 Else;
1131     CustomItem = 'N';
1132 + C7 WkClass = IMPCLS; // C7
1133     01IMDS = IMIMDS;

```

```

1134         010REC = 'S';
1135         01PCLS = IMPCLS;
1136         01STUM = IMUOMS;
1137     EndIf;
1138
1139     SaveDesc = cXML.Request.OrderRequest(orderCounter).
1140         ItemOut(lineCounter).ItemDetail.Description.dsvalue;
1141
1142     SaveDesc = %Subst(SaveDesc:3:298);
1143
1144     // Search/Replace any invalid XML characters in the description
1145
1146     // apostrophe...
1147     CharPos = %Scan(apos: SaveDesc);
1148     DoW CharPos > 0;
1149         SaveDesc = %Replace('\'': SaveDesc: CharPos:
1150             %len(apos));
1151         CharPos = %Scan(apos: SaveDesc: CharPos + 1);
1152     EndDo;
1153
1154     // quote...
1155     CharPos = %Scan(quot: SaveDesc);
1156     DoW CharPos > 0;
1157         SaveDesc = %Replace('\"': SaveDesc: CharPos:
1158             %len(quot));
1159         CharPos = %Scan(quot: SaveDesc: CharPos + 1);
1160     EndDo;
1161
1162     // ampersand...
1163     CharPos = %Scan(amper: SaveDesc);
1164     DoW CharPos > 0;
1165         SaveDesc = %Replace('&': SaveDesc: CharPos:
1166             %len(amper));
1167         CharPos = %Scan(amper: SaveDesc: CharPos + 1);
1168     EndDo;
1169
1170     // less than...
1171     CharPos = %Scan(lt: SaveDesc);
1172     DoW CharPos > 0;
1173         SaveDesc = %Replace('<': SaveDesc: CharPos:
1174             %len(lt));
1175         CharPos = %Scan(lt: SaveDesc: CharPos + 1);
1176     EndDo;
1177
1178     // greater than...
1179     CharPos = %Scan(gt: SaveDesc);
1180     DoW CharPos > 0;
1181         SaveDesc = %Replace('>': SaveDesc: CharPos:
1182             %len(gt));
1183         CharPos = %Scan(gt: SaveDesc: CharPos + 1);
1184     EndDo;
1185
1186
1187     Clear OECMWKR;
1188

```

```

1189         If CustomItem = 'N';
1190             CharPos = %Scan('; ':'SaveDesc);
1191             If CharPos > 0;
1192                 SaveDesc = %Subst(SaveDesc : CharPos + 1 :
1193                     %Len(%Trim(SaveDesc)) - CharPos + 1);
1194             Exsr WriteCmt;
1195             EndIf;
1196         Else;
1197             Exsr WriteCmt;
1198         EndIf;
1199
1200         01SlPrXlt = %XLATE('$*',' ':cXML.Request.
1201             OrderRequest(orderCounter).
1202             ItemOut(lineCounter).ItemDetail.
1203             UnitPrice.Money.dsvalue);
1204
1205 - C17 - 01SlPr = %Dec(01SlPrXlt:13:5);
1206 + C17 // C17 - a malformed price used to halt the program.
1207 + C17 // The quantity conversion above is monitored;
1208 + C17 // this one now matches it.
1209 + C17 Monitor;
1210 + C17 01SlPr = %Dec(01SlPrXlt:13:5);
1211 + C17 On-Error;
1212 + C17 01SlPr = 0;
1213 + C17 EndMon;
1214
1215 01CUOM = 'EA';
1216
1217 // Populate the OEDTWK fields...
1218 01OCTL = LastOrder;
1219 01ORL# = lineCounter;
1220
1221 01MNCD = 'A';
1222 01BLKT = 'N';
1223 01PLT = 5;
1224 01TAXC = 'N';
1225 01LPOV = 'N';
1226 01CPOV = 'N';
1227 01STKL = 'Y';
1228 01LLTC = 'N';
1229 If H1DSHP = 0;
1230     01DSHP = 'N';
1231 Else;
1232     01DSHP = 'Y';
1233 EndIf;
1234 01GNPO = 'N';
1235
1236 // Get the default warehouse for this trading partner...
1237 Chain HDPCust# HDCUSTR;
1238 + C15
1239 + C15 // C15 - a failed chain must not leak the prior
1240 + C15 // customer's ship via, warehouse or location.
1241 + C15 If NOT %Found(HDCUST);
1242 + C15 Clear HDCUSTR;

```

```

1243 +   C15      EndIf;
1244 +   C15
1245 +   C15      // C9 - customer class drives the prefix lead-day
1246 +   C15      //      lookup. No extra I/O, the chain is already here.
1247 +   C15      CustClass = CMCCLS;
1248
1249      If ShipToFnd = 'Y';
1250          01SVSV = CMSVSV;
1251      Else;
1252          01SVSV = CMSV;
1253      EndIf;
1254
1255      // Populate 01SVDS on OEDTWK for HOEPIC pick ticket Ship Via
1256      Chain 01SVSV HDSHPV;
1257      If %Found(HDSHPV);
1258          01SVDS = SVSVDS;
1259      Else;
1260          01SVDS = *Blanks;
1261      EndIf;
1262
1263      01WH   = CMWH#;
1264      01BOAL = *Blanks;
1265      01OCTL = LastOrder;
1266      01LOC# = CMLOC#;
1267      01SHTO = H1SHTO;
1268      01ORST = 'O';
1269
1270
1271      Chain (01ITEM:01WH) HDIWHs;
1272      If %Found(HDIWHs);
1273          01PGRP = IWPGRP;
1274          01PRUM = IWUOMP;
1275          01ORUF = IWPCPP;
1276          01TAXC = IWTXCD;
1277          // Per Bill Busch 10/15/2019 ... ALWAYS USE PLANT 1 for anything
1278          // coming in via XML from Coupa
1279          // 01PLT = IWPLT;
1280          01PLT = 1;
1281      Else;
1282 +   C8      // C8 - a failed chain must not leak the prior
1283 +   C8      //      line's quantities into Avail.
1284 +   C8      IWOHQT = 0;
1285 +   C8      IWQOO  = 0;
1286 +   C8      IWRESQ = 0;
1287          01PGRP = *Blanks;
1288          01PRUM = 'EA';
1289          01PLT = 1;
1290          01ORUF = 1.0000;
1291          01TAXC = STTxCd;
1292      EndIf;
1293
1294      Chain (01PLT:01ITEM) HDIPLT;
1295
1296      // Determine availability for THIS item...
1297      If %Found(HDIPLT);

```



```

1298      Avail = (IWOHQQT + IWQ00) - (IWRESQ + IPCMT0);
1299 +    C4      LeadDays = IPTLT;                      // C4
1300      Else;
1301 +    C4      LeadDays = 0;                          // C4
1302
1303      Alert = 'XMLConfig';
1304      Select;
1305      When ProcessMode = 'T';
1306          AlertSubj = 'Testing Waste Management archive ' +
1307                      'order file ' + %Trim(xmlFile);
1308      When xmlMode = 'T';
1309          AlertSubj = 'Waste Management sent in test order '
1310                      + HWPO#;
1311      Other;
1312          AlertSubj = 'Waste Management Order Alert: Stock' +
1313                      ' Item Not Set Up';
1314      EndSL;
1315      AlertMsg = 'HarrisData Order: '
1316                + %Char(LastOrder) + CrLf;
1317      AlertMsg = %Trim(AlertMsg) + 'Waste Management Order: '
1318                + HWPO# + CrLf;
1319      AlertMsg = %Trim(AlertMsg) + 'Order Line: '
1320                + %Char(O1OrL#) + CrLf;
1321      AlertMsg = %Trim(AlertMsg) + 'Item: '
1322                + O1BItem + CrLf;
1323      AlertMsg = %Trim(AlertMsg) + 'Issue: Configuration'
1324                + CrLf;
1325      AlertMsg = %Trim(AlertMsg) +
1326                'Details: Stock item was not found ' +
1327                'on the HarrisData Item Plant file.';
1328
1329      Exsr Notify;
1330
1331      Avail = 0;
1332
1333      EndIf;

```

```

-    C4 -
-    C4 -
-    C4 -----//
-    C4 -----// Change per Bill on 5/17/2023... add days to item required
-    C4 -----// date based on availability and custom or non custom
-    C4 -----// item... will AddDays will control how many days to add to
-    C4 -----// the current date to determine the calculated required date.
-    C4 -----//
-    C4 -----// Added the use of the UDC table 10/19/2025 because there
-    C4 -----// are some items that aren't following the generic rules.
-    C4 -----// Those items will get the number of days specified in the
-    C4 -----// UDC table added where everything else will fall through
-    C4 -----// the generic logic defined here.
-    C4 -----//
-    C4 -
-    C4 -----// Clear AddDays for each item so CactReqD is accurate
-    C4 -----// per each item.
-    C4 -----AddDays = 0;
-    C4 -

```

```

- C4 ----- //
- C4 ----- // Availability FIRST, then the UDC override (per Bill).
- C4 ----- // If available -> honor the per item UDC override (or the
- C4 ----- // in stock default of 1 day). If short -> fall through to the
- C4 ----- // generic day count rules below.
- C4 ----- //
- C4 -
- C4 ----- If Avail >= 01QOrd;
- C4 -
- C4 ----- // Available -> honor the per item UDC override
- C4 ----- Exec SQL
- C4 ----- Select Coalesce(Cast(UDCDESC1 As Decimal(2,0)), 0)
- C4 ----- Into :UDCDays ;NullInd
- C4 ----- from ProITRG/UDCDETAIL
- C4 ----- Where UDCSYSTEMD = 'ORDERLINES' And UDCCODED = 'REQUIREDDAY'
- C4 ----- And UDCKEY = :01BItem;
- C4 -
- C4 ----- If UDCDays > 0; ----- // on the UDC table
- C4 ----- AddDays = UDCDays;
- C4 ----- Else; ----- // not on the table -> in stock default
- C4 ----- AddDays = 1;
- C4 ----- EndIf;
- C4 -
- C4 ----- Else;
- C4 -
- C4 ----- // Not available -> generic day count rules
- C4 ----- Select;
- C4 -
- C4 ----- When impels = 'DCCU' Or
- C4 ----- impels = 'DCSG';
- C4 -
- C4 ----- // KIT (Digital Cut) product class DCCU / DCSG -> 5 days
- C4 ----- AddDays = 5;
- C4 -
- C4 ----- When impels = 'KITS';
- C4 -
- C4 ----- // KIT (Digital / Screen Print) product class KITS -> 10 days
- C4 ----- AddDays = 10;
- C4 -
- C4 ----- When (%Subst(01BItem:1:1) = '9' And
- C4 ----- %Subst(01BItem:3:1) = ' ') Or
- C4 ----- %Subst(01BItem:1:3) = '3M0';
- C4 -
- C4 ----- // Stock item, NOT in stock -> 5 business days and notify CS
- C4 ----- StockItem = 'Y';
- C4 ----- AddDays = 5;
- C4 -
- C4 ----- Alert = 'XMLAvail';
- C4 ----- If xmlMode = 'T';
- C4 ----- AlertSubj = '*** TEST ***';
- C4 ----- EndIf;
- C4 ----- AlertSubj = %Trim(AlertSubj) + ' ' +
- C4 ----- 'Import Order Alert: STOCK ITEM AVAILABILITY';
- C4 ----- AlertMsg = 'Import Order Customer: ' + xmlRetailer + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '

```

```

- C4 ----- + %Char(LastOrder) + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Import Order: '
- C4 ----- + HWPO# + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Order Line: '
- C4 ----- + %Char(O1OrL#) + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Item: '
- C4 ----- + O1BItem + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Issue: Availability'
- C4 ----- + CrLf;
- C4 ----- AlertMsg = %Trim(AlertMsg) + 'Details: ' +
- C4 ----- %Char(O1QOrd) +
- C4 ----- ' Ordered Only ' + %Char(Avail) +
- C4 ----- ' are available.';
- C4 ----- Exsp Notify;
- C4 -
- C4 ----- Other;
- C4 -
- C4 ----- // Custom / non-stock item -> 10 days
- C4 ----- AddDays = 10;
- C4 -
- C4 ----- EndSl;
- C4 -
- C4 ----- EndIf;
- C4 -
- C4 ----- //
- C4 ----- // Changed per Bill on 05/17/2023... moved calculation
- C4 ----- // for required date down to after availability check.
- C4 ----- //
- C4 -

```

```

1334 + C4 //=====
1335 + C4 // C1/C2 Hardcoded product-class branches (DCCU / DCSG / KITS)
1336 + C4 // and the 9x- / 3M0 stock-name test are removed. The
1337 + C4 // XMLAvail short-stock alert lived inside that block and
1338 + C4 // is removed with it.
1339 + C4 // C4 The required-date day count now comes entirely from the
1340 + C4 // UDC file, in this order:
1341 + C4 // 1. ORDERLINES / REQUIREDAY on the item
1342 + C4 // 2. ORDERLINES / PREFIXRQDD on the item prefix,
1343 + C4 // scoped to this customer's class, or the
1344 + C4 // all-customers key 'SCREEN GRAPHICS'
1345 + C4 // 3. ORDERLINES / PCREQUIRED on the product class
1346 + C4 // 4. HDIPLT.IPTLT, only when above 1 day
1347 + C4 // 5. default of 10
1348 + C4 // In stock always returns 1 business day.
1349 + C4 // CalcReqD skips weekends and HOLIDAY rows, so every
1350 + C4 // value set below is a business-day count.
1351 + C4 //=====
1352 + C4
1353 + C4 AddDays = 0;
1354 + C4
1355 + C4 If Avail >= O1QOrd;
1356 + C4
1357 + C4 // In stock -> next business day. No lookup: an item with
1358 + C4 // exact or more available is always 1 day.
1359 + C4 AddDays = 1;

```

```

1360 + C4
1361 + C4 Else;
1362 + C4
1363 + C4 // ReqDays is zeroed first and each tier runs only while it is
1364 + C4 // still zero, so a no-row condition (SQLCODE 100) can never
1365 + C4 // carry a prior line's value forward.
1366 + C4 ReqDays = 0;
1367 + C4
1368 + C4 // 1. exact item
1369 + C4 Exec SQL
1370 + C4     Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1371 + C4     Into :ReqDays
1372 + C4     From ProITRG/UDCDETAIL
1373 + C4     Where UDCSYSTEMD = 'ORDERLINES'
1374 + C4     And UDCCODED = 'REQUIREDAY'
1375 + C4     And UDCKEY = :01BItem;
1376 + C4
1377 + C4 // 2. item prefix, scoped by customer class. UDCKEY holds
1378 + C4 // either a 2 character HDCCLS class code, which must match
1379 + C4 // this customer's CMCCLS, or the literal 'SCREEN GRAPHICS'
1380 + C4 // which applies to every customer. A customer specific row
1381 + C4 // outranks 'SCREEN GRAPHICS'; among equals the longest
1382 + C4 // prefix wins.
1383 + C4 If ReqDays = 0;
1384 + C4     Exec SQL
1385 + C4         Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1386 + C4         Into :ReqDays
1387 + C4         From ProITRG/UDCDETAIL
1388 + C4         Where UDCSYSTEMD = 'ORDERLINES'
1389 + C4         And UDCCODED = 'PREFIXRQDD'
1390 + C4         And :01BItem Like Concat(RTrim(UDCINDEXED),'%')
1391 + C4         And (UDCKEY = :CustClass Or
1392 + C4             UDCKEY = 'SCREEN GRAPHICS')
1393 + C4         Order By Case When UDCKEY = 'SCREEN GRAPHICS'
1394 + C4             Then 1 Else 0 End,
1395 + C4             Length(RTrim(UDCINDEXED)) Desc
1396 + C4         Fetch First 1 Row Only;
1397 + C4     EndIf;
1398 + C4
1399 + C4 // 3. product class, taken from the guarded HDIMST chain
1400 + C4 If ReqDays = 0;
1401 + C4     Exec SQL
1402 + C4         Select Coalesce(Cast(UDCDESC1 As Decimal(3,0)),0)
1403 + C4         Into :ReqDays
1404 + C4         From ProITRG/UDCDETAIL
1405 + C4         Where UDCSYSTEMD = 'ORDERLINES'
1406 + C4         And UDCCODED = 'PCREQUIRED'
1407 + C4         And UDCKEY = :WkClass;
1408 + C4     EndIf;
1409 + C4
1410 + C4 Select;
1411 + C4 When ReqDays > 0;
1412 + C4     AddDays = ReqDays;
1413 + C4
1414 + C4 When LeadDays > 1;

```

```

1415 + C4 // Plant lead time. IPTLT is DECIMAL(5,1), so round a partial
1416 + C4 // day up: a line cannot ship in part of a business day.
1417 + C4 // A value of 1 or less is a ship-from-stock figure and is
1418 + C4 // meaningless for a short line, so it falls through to the
1419 + C4 // default rather than promising next day on a short order.
1420 + C4 AddDays = %Int(LeadDays);
1421 + C4 If LeadDays > AddDays;
1422 + C4 AddDays += 1;
1423 + C4 EndIf;
1424 + C4
1425 + C4 Other;
1426 + C4 AddDays = 10;
1427 + C4
1428 + C4 EndS1;
1429 + C4
1430 + C4 EndIf;
1431 Exsr CalcReqD;
1432
1433 //-----
1434
1435
1436 Chain (01Plt:1:01ITEM) HDMCMM;
1437 If %Found(HDMCMM);
1438 01UCC1 = CMUCC1;
1439 01UCC2 = CMUCC2;
1440 01UCC3 = CMUCC3;
1441 01UCC4 = CMUCC4;
1442 01UCC5 = CMUCC5;
1443 01COST = 01UCC1 + 01UCC2 + 01UCC3 + 01UCC4 + 01UCC5;
1444 EndIf;
1445
1446 01LIST = 01QORD * 01S1Pr;
1447 OrdTotal += 01List;
1448 01QAFG = '1';
1449 01UDA1 = %Subst(giftMSG:1:15);
1450 01UDA1 = %XLate(Lo:Up:01UDA1);
1451 01UDA2 = %Subst(giftMSG:16:15);
1452 01UDA2 = %XLate(Lo:Up:01UDA2);
1453 01UDA3 = %Subst(giftMSG:31:15);
1454 01UDA3 = %XLate(Lo:Up:01UDA3);
1455 01UDA4 = %Subst(giftMSG:45);
1456 01UDA4 = %XLate(Lo:Up:01UDA4);
1457
1458 // 01RqDt is calculated in CalcReqD. It was being overwritten
1459 // with the incorrect date here (LastDt1Date) so it was removed
1460 // and verified with Bill on the phone in a debug on 6/13/2023.
1461
1462 Write OEDTWKR;
1463 + C13 ProcessedItems += 1; // C13
1464
1465 If ProcessMode = 'L';
1466 XDOBAT = HWPO#;
1467 XDSNDR = cXML.Header.From.Credential.Identity;
1468 // XDXORD = %Dec(XMLOrderID:11:0);
1469 XDXORD = 0;

```

```

1470      XDPO      = HWPO#;
1471      XDHDOR = LastOrder;
1472      XDOLIN = lineCounter;
1473      XDLNIID = %Char(lineCounter);
1474      XDMLin = %Char(cXML.Request.OrderRequest(orderCounter).
1475                  ItemOut(lineCounter).lineNumber) ;
1476
1477      XDQty0 = %Dec(%XLATE('$*',' ':cXML.Request.
1478                  OrderRequest(orderCounter).
1479                  ItemOut(lineCounter).
1480                  quantity) : 13 : 4);
1481
1482      XDUOM = 'EA';
1483      XDDESC = O1IMDS;
1484      XDMSKU = cXML.Request.OrderRequest(orderCounter).
1485              ItemOut(lineCounter).ItemID.SupplierPartID;
1486
1487      XDUCST = O1S1Pr;
1488      XDLSHP = O1SvSv;
1489      XDLSHP = %XLate(Lo:Up:XDLSHP);
1490      XDEShp = %char>LastDt1Date:*ymd0);
1491
1492      Write XMLOEDTR;
1493      EndIf;
1494
1495      EndFor;
1496 +   C13
1497 +   C13      //=====
1498 +   C13      // C13 Order arrived with no detail lines. Alert only. The order
1499 +   C13      // IS still created: execution falls straight through to the
1500 +   C13      // H1RqDt fallback, Write OEHDWKR, the tax call, the data queue
1501 +   C13      // and the XMLOEHD write, exactly as it does today.
1502 +   C13      // Do NOT add LeaveSR here.
1503 +   C13      //=====
1504 +   C13      If ProcessedItems = 0;
1505 +   C13
1506 +   C13          Alert      = 'XMLNODTL';
1507 +   C13          AlertSubj = *Blanks;
1508 +   C13          If xmlMode = 'T';
1509 +   C13              AlertSubj = '*** TEST ***';
1510 +   C13          EndIf;
1511 +   C13          AlertSubj = %Trim(AlertSubj) + ' ' +
1512 +   C13                  'Imported Order Alert: Order has no detail';
1513 +   C13          AlertMsg = 'Customer: ' + %Trim(xmlRetailer) + CrLf;
1514 +   C13          AlertMsg = %Trim(AlertMsg) + 'Imported File: ' +
1515 +   C13                  %Trim(ifsFile) + CrLf;
1516 +   C13          AlertMsg = %Trim(AlertMsg) + 'Imported Order: ' +
1517 +   C13                  %Trim(HWPO#) + CrLf;
1518 +   C13          AlertMsg = %Trim(AlertMsg) + 'Issue: No Items Ordered' + CrLf;
1519 +   C13          Exsr Notify;
1520 +   C13
1521 +   C13      EndIf;
1522 +   C13      //=====
1523
1524      // Notify Bill Busch if the order didn't have anything to process

```

```

1525 // in it. Skip the rest of the processing and do NOT write to
1526 // OEHDWK
1527 // If ProcessedItem = 0;
1528 // Notify Bill via email. End processing and move the file
1529 // to the archive
1530 // Alert = 'XMLNODTL';
1531 // If xmlMode = 'T';
1532 // AlertSubj = '*** TEST ***' ;
1533 // EndIf;
1534 // AlertSubj = %Trim(AlertSubj) + ' ' +
1535 // 'Imported Order Alert: Order has no detail';
1536 // AlertMsg = 'Customer: ' + xmlRetailer + CrLf;
1537 // AlertMsg = 'Imported File: ' + %Trim(ifsFile) + CrLf;
1538 // AlertMsg = %Trim(AlertMsg) + 'Imported Order: '
1539 // + HWPO# + CrLf;
1540 // AlertMsg = %Trim(AlertMsg) + 'Issue: No Items Ordered'
1541 // + CrLf;
1542 // Exsr Notify;
1543
1544 // Exit the subroutine and let the exitProgram routine archive
1545 // the file and exit to process the next order.
1546 // LeaveSR;
1547
1548 // EndIf;
1549
1550
1551 // No line set a date -> fall back to the next business day
1552 If H1RqDt = 0;
1553 H1RqDt = %Dec(%Char(LastDt1Date:*CYMD0):7:0);
1554 EndIf;
1555
1556 // Write the OEHDWK record with the order total...
1557
1558 H1OCTL = LastOrder;
1559 H1ORD# = LastOrder;
1560 H1MNCD = 'A';
1561 H1BLTO = HDPCust#;
1562
1563 Chain H1BLTO HDCUSTR;
1564 + C16
1565 + C16 // C16 - a failed chain must not leave the previous
1566 + C16 // customer's values on the order header.
1567 + C16 If NOT %Found(HDCUST);
1568 + C16 Clear HDCUSTR;
1569 + C16 EndIf;
1570
1571 H1Bank = 7;
1572 H1Loc# = CMLoc#;
1573
1574 H1SLSM = CMSLSM;
1575
1576 Clear HDTAXDR;
1577 Chain (STState: 0: 0: 0: 0: 0) HDTAXDR;
1578
1579 H1STTX = TDSTCD;

```

```

1580         If ShipToFnd <> 'Y';
1581             H1CTXC = CMCTXC;
1582         Else;
1583             H1CTXC = STTxCd;
1584         EndIf;
1585         H1CNTX = CMCNTC;
1586         H1CTTX = CMCTTC;
1587         H1LOC1 = CMLOC1;
1588         H1LOC2 = CMLOC2;
1589         H1LOC3 = CMLOC3;
1590         H1BDTE = %dec(%char(%date(%Subst(HWOrdDate:1:10):*iso-)*cymd0):7:0);
1591         H1ORTY = CMORTY;
1592         H1SVSV = CMSV;
1593         H1ORRF = HWPO#;
1594         H1CTRM = CMCTRM;
1595         If ShipToFnd = 'Y';
1596             H1SVSV = CMSVSV;
1597         EndIf;
1598
1599         // Pull the Ship Via Description
1600         Chain H1SVSV HDShPV;
1601         If %Found(HDShPV);
1602             H1SVDS = SVSVDS;
1603         Else;
1604             H1SVDS = *Blanks;
1605         EndIf;
1606
1607         H1BOAL = 'N';
1608         H1TOT = OrdTotal;
1609         H1FBCD = 'OR';
1610
1611         // Set Order Flags (Pulled directly from HOEOEM)
1612
1613         FGORTY = CMORTY;
1614
1615     /End-Free
1616     //
1617 C      KFG          KLIST
1618 C          KFLD          APID
1619 C          KFLD          FGORTY
1620     //
1621 /FREE
1622     CLEAR DFL;
1623     SETLL KFG HDORFGR;
1624     DOU 1=2;
1625         READE KFG HDORFGR;
1626         IF %EOF(HDORFG);
1627             LEAVE;
1628         ENDIF;
1629 /END-FREE
1630 C          Z-ADD      FGFGFG      F
1631 C          MOVE      FGFGOP      DFL(F)
1632 /FREE
1633     ENDDO;
1634     //

```



```

1635 /END-FREE
1636 C          MOVEA      DFL          H10EF
1637 //
1638 /Free
1639
1640 // H1RQDT (Header Required Date) is derived beginning in NxthDOrd
1641 // where it is initially assigned the next day but can be overwritten
1642 // in CalcReqD when each line item is calculated because Bill wants
1643 // it to be the EARLIES ship date of any item on the order, NOT THE
1644 // LATEST... per our phone conversation during debug on 6/13/2023.
1645
1646 Write OEHDWKR;
1647
1648
1649
1650 // Calculate taxes and add them to the order...
1651 xmlOCTL   = LastOrder;
1652 xmlFRT    = 0.00;
1653 xmlCHRG   = 0.00;
1654 xmlUPDTOT = 'R';
1655 HOEUMC_W(HexHnd : xmlDBID : xmlOCTL : xmlFRT : xmlCHRG : xmlUPDTOT);
1656
1657
1658 // Notify customerservice@screen-graphics.com if this order had any
1659 // "CUSTOM ITEMS" on it. Specify all the normal detail information
1660 // but specify each "CUSTOM ITEM" detail line by number with its
1661 // full description, quantity, price, and extended price.
1662
1663 If CustomItem = 'Y';
1664     Exsr AlertCustom;
1665 EndIf;
1666
1667
1668 // Submit the order to the Data Queue...
1669 OQTYP = 'ORVW';
1670 QTURN = 0;
1671 QQORD# = LastOrder;
1672 QQOCTL = LastOrder;
1673 QQTSTP = %TimeStamp();
1674 QQTSUS = %SUBST(XMLRetailer:1:10);
1675 QQTSWS = 'XML850R_AR';
1676 QQPGM = 'HOEOEM';
1677 QSNDDTAQ(DQ#NAM:DQ#LIB:DQ#LEN:DTAQ);
1678
1679 If ProcessMode = 'L';
1680     // Now write the order to the XML Orders files...
1681     Clear XMLOEHDR;
1682
1683     XHFILE = XMLFILE;
1684     XHOBAT = XMLOrderID;
1685     XHPTNR = XMLRetailer;
1686     XHSNDR = 'Amerigas';
1687     XHTRNS = Transact;
1688     // XHXORD = %Dec(XMLOrderID:11:0);
1689     XHXORD = 0;

```

```

1690      XHPO      = HWPO#;
1691      XHHDOR = LastOrder;
1692      XHOLNT = HWLinCnt;
1693      XHPOTY = HWPOType;
1694      XHGFTM = HWGiftMSG;
1695      XHODTE = HWOrdDate;
1696      XSHSCD = HWShipCd;
1697      XHCNTL = HWControl;
1698      XHSLDV = HWSlsDiv;
1699      XHCORD = XMLOrderID;
1700      XHBCNT = HWBuyCntrct;
1701      XHMDPT = HWMerchDpt;
1702      XHRQSD = %EditC(H1RqDt:'X');
1703      // Capture Ship-To Address Only...
1704      XSHSPP      = HWShpToID;
1705      XHPPNM1      = STName1;
1706      XHPP1A1      = STAdr1;
1707      XHPP1A2      = STAdr2;
1708      XHCTY1      = STCity;
1709      XHST1      = STState;
1710      XHCNT1      = STCountry;
1711      XHZIP1      = STZIP;
1712      XHEML1      = STEmail;
1713      XHDPH1      = STPhone;
1714      XHBLPP = HWBillToID;
1715      XHCUPP = HWCustID;
1716      XHIVPP = HWInvToID;
1717      XHEML1 = STEmail;
1718
1719      Write XMLOEHDR;
1720
1721      EndIf;
1722
1723      EndSr;
1724      //-----*
1725
1726
1727
1728      //-----*
1729      // WriteCmt will handle writing detail line item comments to an *
1730      // order. *
1731      //-----*
1732
1733      BegSr WriteCmt;
1734
1735      For x = 1 to 5;
1736          04CSEQ = x;
1737          Select;
1738              When x = 1;
1739                  04CMNT = %Subst(SaveDesc : 1 : 60);
1740              When x = 2;
1741                  04CMNT = %Subst(SaveDesc : 61 : 60);
1742              When x = 3;
1743                  04CMNT = %Subst(SaveDesc : 121 : 60);
1744              When x = 4;

```

```

1745         O4CMNT = %Subst(SaveDesc : 181 : 60);
1746     When x = 5;
1747         O4CMNT = %Subst(SaveDesc : 241 : 60);
1748     EndSl;
1749
1750     If O4CMNT <> *Blanks;
1751         O4OCTL = LastOrder;
1752         O4ORL# = lineCounter;
1753         O4BLN# = 0;
1754         O4DOCT = 'ACK';
1755         O4CSR = *Blanks;
1756
1757         Chain (O4OCTL:O4ORL#:O4BLN#:O4DOCT:O4CSEQ) OECMWKR;
1758         If NOT %Found(OECMWK);
1759             Write OECMWKR;
1760         EndIf;
1761     EndIf;
1762 EndFor;
1763
1764 EndSr;
1765 //-----*
1766
1767
1768
1769
1770 //-----*
1771 /* AlertCustom notifies customerservice@screen-grpahics.com of *
1772 /* the orders with 'CUSTOM ITEM's on them and the details about *
1773 /* those lines. *
1774 //-----*
1775 BegSr AlertCustom;
1776
1777 Select;
1778 When ProcessMode = 'T';
1779     AlertSubj = 'Testing Waste Management archived ' +
1780         'order file ' + %Trim(xmlFile) + ' with ' +
1781         'a custom item.';
1782 When xmlMode = 'T';
1783     AlertSubj = 'Waste Management sent a test order in ' +
1784         'file ' + %Trim(xmlFile) + ' with a ' +
1785         'custom item.';
1786 Other;
1787     AlertSubj = 'Waste Management has at least one ' +
1788         'custom item in order ' + %Trim(HWPO#) +
1789         ' in file ' + %Trim(xmlFile);
1790 EndSl;
1791 AlertMsg = %Trim(AlertMsg) + 'HarrisData Order: '
1792     + %Char(LastOrder) + CrLf;
1793
1794 // Now pull the order from OEDTWK, each 'CUSTOM ITEM' line,
1795 // and report all the details about those lines for customer service.
1796 Exec SQL
1797     Declare Custom Cursor for
1798         Select O1ORL#, O1RQDT, O1IMDS, O4CMNT, O1QORD, O1SLPR
1799             from OEDTWK Left Outer Join OECMWK on

```

```

1800         01OCTL = 04OCTL And 01ORL# = 04ORL# And
1801         04DOCT = 'INT' And 04CSEQ = 1
1802     Where 01ITEM = 'CUSTOM ITEM' and
1803         01OCTL = :LastOrder ;
1804
1805     Exec SQL
1806         Open Custom;
1807
1808     Exec SQL
1809         Fetch Next from Custom Into :CustOrdLn, :CustRqDt,
1810                                     :CustDesc, :CustCmt,
1811                                     :CustQOrd, :CustSlPr;
1812
1813     pass = 0;
1814
1815     DoW SQLCod = 0;
1816
1817     pass += 1;
1818
1819     If pass > 1;
1820         AlertMsg = %Trim(AlertMsg) + CrLf;
1821     EndIf;
1822
1823     AlertMsg = %Trim(AlertMsg) + 'Order Line: '
1824               + %Char(CustOrdLn) + CrLf;
1825     AlertMsg = %Trim(AlertMsg) + 'Required: '
1826               + %Char(CustRqDt) + CrLf;
1827     AlertMsg = %Trim(AlertMsg) + 'Description: '
1828               + %Trim(CustDesc) + %Trim(CustCmt) +
1829               CrLf;
1830     AlertMsg = %Trim(AlertMsg) + 'Qty: ' +
1831               %Char(CustQOrd) + CrLf;
1832     AlertMsg = %Trim(AlertMsg) + 'Price: ' +
1833               %Char(CustSlPr) + CrLf +
1834               %Char(CustSlPr) + CrLf ;
1835     AlertMsg = %Trim(AlertMsg) + 'Extended Price: ' +
1836               %Char(CustQOrd * CustSlPr);
1837
1838     Exec SQL
1839         Fetch Next from Custom Into :CustOrdLn,
1840                                     :CustRqDt, :CustDesc, :CustCmt,
1841                                     :CustQOrd, :CustSlPr;
1842
1843     EndDo;
1844
1845     Exec SQL
1846         Close Custom;
1847
1848     Exsr Notify;
1849
1850     EndSr;
1851     //-----*
1852
1853

```

```

1854 //-----*
1855 /* exitProgram archives the order processed and terminates the *
1856 /* program upon completion *
1857 //-----*
1858 BegSr exitProgram;
1859
1860 // Archive the order file...
1861
1862 Cmd = 'CPY OBJ(' + '''' + %Trim(filename) + '''' + ') ' +
1863 'TODIR(' + '''' + '/orders/archive inbound/' +
1864 %Trim(XMLRetailer) + '/orders' + '''' + ') ' +
1865 'REPLACE(*YES) OWNER(*NEW)';
1866 CmdLen = %Len(%Trim(Cmd));
1867 ExecCmd (Cmd:CmdLen);
1868
1869 Cmd = 'RMVLNK OBJLNK(' + '''' + %Trim(filename) + '''' + ')';
1870 CmdLen = %Len(%Trim(Cmd));
1871 ExecCmd (Cmd:CmdLen);
1872
1873 EndSr;
1874 //-----*
1875
1876
1877
1878 // -----//
1879 // NxthDOrd subroutine will pull the last order number Harris //
1880 // used and update it for the new cXML order. It will also //
1881 // write to the XML cross reference file of orders. //
1882 // -----//
1883 BegSR NxthDOrd;
1884
1885 Exec SQL
1886 Select CELORD into :LastOrder from OECtrl;
1887
1888 LastOrder += 1;
1889
1890 Exec SQL
1891 Update OECTRL set CELORD = :LastOrder;
1892
1893 CurDate = %Date();
1894 CurTime = %Time();
1895
1896 Exec SQL
1897 Insert into XMLORDS (XCCOORD, XHDSORD, XPROCISO, XPROCHMS)
1898 Values (:XMLOrderID, :LastOrder, :CurDate, :CurTime);
1899
1900 // Calculate the required date for the order
1901 TempDate = %Date() + %Days(1);
1902
1903 // *Mon = 0 ... Sun = 6
1904 Day_Of_Week = %rem(%diff(%date() +
1905 %Days(1):d'0001-01-01':*days):7);
1906 If Day_of_Week = 5; //Saturday
1907 TempDate = %Date() + %Days(3);
1908 EndIf;

```

```

1909      If Day_of_Week = 6;    // Sunday
1910          TempDate = %Date() + %Days(2);
1911      EndIf;
1912
1913      LastDt1Date = TempDate;
1914      // Seed header date to zero; CalcReqD sets it to the earliest
1915      // line required date. Empty orders fall back after the loop.
1916      H1RqDt = 0;
1917
1918      EndSR;
1919      // -----//
1920
1921      // -----//
1922      // CalcReqD subroutine will calculate the required date for    //
1923      // individual detail line item based on Bill's criteria of    //
1924      // the number of days to add based on the item ordered. It    //
1925      // will also take into account any holidays and weekends and  //
1926      // not include those in the data calculations.                //
1927      // -----//
1928      BegSR CalcReqD;
1929
1930      TodayISO = %Date();
1931      TempDate = TodayISO;
1932
1933      // Add AddDays *business* days: step one calendar day at a
1934      // time, skipping Saturdays, Sundays, and holidays until
1935      // AddDays working days have been counted. The order date
1936      // itself is not counted (Tue + 1 business day = Wed).
1937      DoW AddDays > 0;
1938
1939      TempDate = TempDate + %Days(1);
1940
1941      // Mon = 0 ... Sun = 6
1942      Day_of_Week = %rem(%diff(TempDate:
1943          d'0001-01-01':*days):7);
1944      If Day_of_Week = 5 Or Day_of_Week = 6;    // Sat / Sun
1945          Iter;
1946      EndIf;
1947
1948      // Company holiday?
1949      Exec SQL
1950          Select Count(*) Into :DaysOff
1951          From PROITRG/UDCDetail
1952          Where Date(UDCIndexed) = :TempDate
1953          And UDSystemD = 'HOLIDAY';
1954      If DaysOff > 0;
1955          Iter;
1956      EndIf;
1957
1958      AddDays -= 1;    // a real working day
1959
1960      EndDo;
1961
1962      // TempDate builds the actual 01RqDt...
1963      01RqDt = %Dec(%Char(TempDate:*CYMD0):7:0);

```

```

1964
1965      // Header required date = earliest of the line required dates
1966      If H1RqDt = 0 Or O1RqDt < H1RqDt;
1967          H1RqDt = O1RqDt;
1968      EndIf;
1969
1970 EndSR;
1971
1972
1973      // -----//
1974      // ClearOrd subroutine will clear all the processing for this //
1975      // PO# since the order is rejected due to a bad item.          //
1976      // -----//
1977 BegSR ClearOrd;
1978
1979      Exec SQL
1980          Delete From OEHDWK Where H1ORRF = :HWPO#;
1981
1982      Exec SQL
1983          Delete From OEDTWK Where O1OCTL = :LastOrder;
1984
1985      // Notify users of the cancellation...
1986      Exsr Notify;
1987
1988 EndSR;
1989      // -----//
1990
1991
1992
1993      // -----//
1994      // Notify subroutine will send notification of rejection for //
1995      // the order to the appropriate individuals.                  //
1996      // -----//
1997 BegSR Notify;
1998
1999      Exec SQL
2000      Declare Emails Cursor for
2001          Select UDCDESC1 from PROITRG/UDCDetail
2002          Where UDCSYSTEMD = 'EMAIL' and UDCCODED = :Alert;
2003
2004      Exec SQL
2005          Open Emails;
2006
2007      Exec SQL
2008          Fetch Next from Emails into :Distribution;
2009
2010      DoW SQLCod = 0;
2011
2012      // Actual email routine
2013      from = 'hdfiles@screen-graphics.com';
2014      to   = Distribution;
2015      replyTo = *Blanks;
2016      subject = AlertSubj;
2017      text = AlertMsg;
2018

```

```

2019      Monitor;
2020          email = RPGMail_new();
2021          RPGMail_addAddress(email:'FROM':from: ' ');
2022          RPGMail_addAddress(email:'TO':to: ' ');
2023          RPGMail_setSubject(email:subject);
2024          RPGMail_setBodyText(email: text: 'text/plain');
2025          RPGMail_setSMTPHost(email: 'MAILGATE');
2026          RPGMail_setSMTPPort(email: '25');
2027          RPGMail_send(email);
2028      On-error;
2029          error = Error_catch();
2030          // Don't swallow the failure - log it to the job log.
2031          LogTxt = 'XML850R_WM: e-mail send failed for ' + %Trim(to) +
2032                  ' (alert ' + %Trim(Alert) + '). ' +
2033                  %Trim(error.text);
2034          SndPgmMsg( *blanks : *blanks : LogTxt
2035                   : %Len(%Trim(LogTxt)) : '*DIAG' : '*PGMBDY' : 1
2036                   : MsgKey : ApiErr );
2037      EndMon;
2038
2039      Exec SQL
2040          Fetch Next from Emails into :Distribution;
2041
2042      EndDo;
2043
2044      Exec SQL
2045          Close Emails;
2046
2047      + C20
2048      + C20          // C20 - reset so the next alert cannot inherit this
2049      + C20          //      one's subject or body text.
2050      + C20          Clear Alert;
2051      + C20          Clear AlertSubj;
2052      + C20          Clear AlertMsg;
2053      EndSR;
2054      // -----//

```